Divide & Conquer Strategy: PROJECT_ALPHA

Version: 1.0.4 | **Environment:** DEVELOPMENT | **Security Level:** HIGH **Core Tech:** Node.js / Express (Port 8080), PostgreSQL

Based on the blueprint, the project naturally splits into two distinct domains: **Security/Infrastructure** and **Catalog/Business Logic**. Here is how you and your partner should divide the work.

Developer A: The "Security & Infrastructure" Lead

Focus: App configuration, Auth0/Clerk integration, Admin user management, and API protection.

1. Project Setup & Config

- Initialize the Node.js/Express server running on `PORT: 8080`.
- Implement the strict security features required in the blueprint config:
  - Enable **Strict SSL**.
  - Configure **CORS** (Allowing requests from the Public Storefront and Admin Dashboa
  - Implement **Rate Limiting**.

2. Database & Auth (Components `c4` & `c5`)

- Create the PostgreSQL connection pool.
- **Table Ownership:** Build the `admin` table (`id` , `email` , `password_hash` , `role` , `last_login`).
- **Auth Provider:** Integrate the chosen provider (**Auth0 or Clerk**) using OAuth 2.0 + JWT

3. The Core Middleware

- Build the `verifyAdminJWT` middleware.
- **The Goal:** Every route starting with `/api/v1/admin/` must pass through this middlew to validate the JWT from Auth0/Clerk before hitting the database.

Developer B: The "Catalog & APIs" Lead

Focus: PostgreSQL schema for products, categories, and building the 8 specific REST endpoints.

1. Database Schema (Tables `t2` & `t3`)

- Build the `categories` table (`id` , `name`).
- Build the `products` table (`id` , `created_at` , `name` , `current_price` , `available` , `new_price` , `description` , `quantity`).
- Establish the Foreign Key relationship between `products.category` and `categories`

2. The Public API (Unsecured)

- Build endpoint `e1` : **GET** `/api/v1/public/products`
  - *Rule:* No authentication required.
  - *Logic:* Fetch available products to display on the Public Storefront (Component `c1`

3. The Admin APIs (Secured)

Build the CRUD endpoints that the Admin Dashboard (Component `c2`) will use. *Note: Developer B will build these routes, but Developer A will protect them.*

- **Category Management:**
  - **GET** `/api/v1/admin/categories` (Fetch all)
  - **POST** `/api/v1/admin/categories` (Create new)
  - **DELETE** `/api/v1/admin/categories` (Remove)
- **Product Management:**
  - **POST** `/api/v1/admin/products` (Create new product with category mapping)
  - **GET** `/api/v1/admin/products` (Fetch products for admin view)
  - **PUT** `/api/v1/admin/products` (Modify pricing, availability, etc.)
  - **DELETE** `/api/v1/admin/products` (Remove product)

Phase 3: The Integration Phase (Working Together)

Because of this split, you can both work simultaneously without editing the same files. Whe you are both ready to integrate:

1. **The Handshake:** Developer A takes their `verifyAdminJWT` middleware and wraps it around Developer B's `/api/v1/admin/*` router.
2. **Testing:** * Attempt to hit the `GET /api/v1/public/products` route without a token (Should succeed with `200 OK`).
   - Attempt to hit `POST /api/v1/admin/products` without a token (Should fail with `4 Unauthorized`).

- Log in via Auth0/Clerk, get the JWT, and hit `POST /api/v1/admin/products` (Sho succeed with `201 Created`).

By sticking to this plan, Developer A guarantees the app is locked down and configured to "HIGH" security standards, while Developer B guarantees the database relations and JSON responses match the blueprint exactly.